


VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

# LAB TASK 1

BECE406E

FPGA System Design Lab

| | |
|---|---|
| **NAME** | KAARMUKILAN A G |
| **REGISTRATION NO** | 23BEC0137 |
| **SLOT** | L33+L34 |
| **FACULTY NAME** | AARTHY M |

## AIM:

i) Design and verify Full adder using gate level/ structural modelling.
ii) Using the designed full adder, design a 4-bit Ripple Carry Adder (RCA) and verify the same using testbench.
iii) Using the RCA designed in question ii) and few logic gates, design a 4x3 multiplier and verify the same using testbench.
iv) Implement the 4x3 multiplier in DE2-115 Board using Intel Quartus Prime.
v) Attach Screenshots of code and outputs of 4-bit sequential multiplier

## SOFTWARE DETAILS:

Functional Simulation: Modelsim
Synthesis & Implementation: Intel Quartus Prime 23.1

## HARDWARE DETAILS:

Family: Cyclone IV E
Package: FBGA
Pin count: 780

Core speed grade – 7
Device - EP4CE115F29C7

## RTL CODE and TESTBENCHES:

### FULL ADDER

```
module FA (A, B, Cin, S, Cout);
    input  A, B, Cin;
    output S, Cout;
    wire AxorB;
    wire AB, BCin, ACin;
    // Sum
    xor G1(AxorB, A, B);
    xor G2(S, AxorB, Cin);
    // Carry
    and G3(AB,   A,    B);
    and G4(BCin, B,  Cin);
    and G5(ACin, A,  Cin);

   or G6 (Cout, AB, BCin, ACin);
endmodule
//23BEC0137 KAARMUKILAN A G
`timescale 1ns/1ps
module FA_tb;
    reg A,B,Cin;
    wire S, Cout;
    FA dut (A,B,Cin,S,Cout);
    initial begin
        A = 0; B = 0; Cin = 0; #10;
        A = 0; B = 0; Cin = 1; #10;
        A = 0; B = 1; Cin = 0; #10;
        A = 0; B = 1; Cin = 1; #10;
        A = 1; B = 0; Cin = 0; #10;
        A = 1; B = 0; Cin = 1; #10;
        A = 1; B = 1; Cin = 0; #10;
        A = 1; B = 1; Cin = 1; #10;
    end
endmodule
```

```
module FA (A, B, Cin, S, Cout);
    input  A, B, Cin;
    output S, Cout;
    wire AxorB;
    wire AB, BCin, ACin;
    // Sum
    xor G1(AxorB, A, B);
    xor G2(S, AxorB, Cin);
    // Carry
    and G3(AB,   A,    B);
    and G4(BCin, B,  Cin);
    and G5(ACin, A,  Cin);

   or G6 (Cout, AB, BCin, ACin);
endmodule
//23BEC0137 KAARMUKILAN A G
```

```
`timescale 1ns/1ps
module FA_tb;
    reg A,B,Cin;
    wire S, Cout;
    FA dut (A,B,Cin,S,Cout);
    initial begin
        A = 0; B = 0; Cin = 0; #10;
        A = 0; B = 0; Cin = 1; #10;
        A = 0; B = 1; Cin = 0; #10;
        A = 0; B = 1; Cin = 1; #10;
        A = 1; B = 0; Cin = 0; #10;
        A = 1; B = 0; Cin = 1; #10;
        A = 1; B = 1; Cin = 0; #10;
        A = 1; B = 1; Cin = 1; #10;
    end
endmodule
```

**RCA_4BIT**

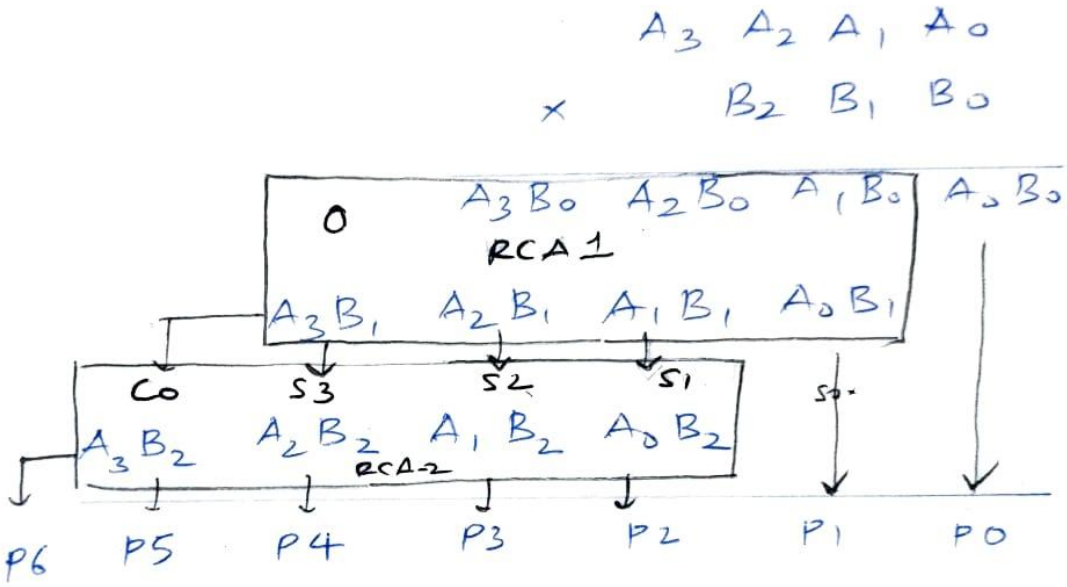
```
module RCA_4 (
    input  [3:0] A,
    input  [3:0] B,
    input        cin,
    output [3:0] Sum,
    output       Cout
);
    wire c0, c1, c2;
    // FA(A, B, Cin, S, Cout)
    FA FA0 (A[0], B[0], cin,  Sum[0], c0);
    FA FA1 (A[1], B[1], c0,   Sum[1], c1);
    FA FA2 (A[2], B[2], c1,   Sum[2], c2);
    FA FA3 (A[3], B[3], c2,   Sum[3], Cout);
endmodule //23BEC0137 KAARMUKILAN A G
`timescale 1ns/1ps
module RCA_4_tb;
    reg  [3:0] A, B;
    reg        cin;
    wire [3:0] Sum;
    wire       Cout;
    RCA_4 dut (A, B, cin, Sum, Cout);
    initial begin
        //  3 + 5 = 8
        A   = 4'b0011;B= 4'b0101;cin = 1'b0; #10;
        //  15 + 1 = 16
        A   = 4'b1111;B   = 4'b0001;cin = 1'b0;  #10;
        //  7 + 8 + 1 = 16
        A   = 4'b0111; B   = 4'b1000;cin = 1'b1;  #10;
        $finish;
    end
endmodule
```

```
module RCA_4 (
    input  [3:0] A,
    input  [3:0] B,
    input        cin,
    output [3:0] Sum,
    output       Cout
);
    wire c0, c1, c2;
    // FA(A, B, Cin, S, Cout)
    FA FA0 (A[0], B[0], cin,  Sum[0], c0);
    FA FA1 (A[1], B[1], c0,   Sum[1], c1);
    FA FA2 (A[2], B[2], c1,   Sum[2], c2);
    FA FA3 (A[3], B[3], c2,   Sum[3], Cout);
endmodule //23BEC0137 KAARMUKILAN A G
```

```
`timescale 1ns/1ps
module RCA_4_tb;
    reg  [3:0] A, B;
    reg        cin;
    wire [3:0] Sum;
    wire       Cout;
    RCA_4 dut (A, B, cin, Sum, Cout);
    initial begin
        //  3 + 5 = 8
        A   = 4'b0011;B= 4'b0101;cin = 1'b0; #10;
        //  15 + 1 = 16
        A   = 4'b1111;B   = 4'b0001;cin = 1'b0;  #10;
        //  7 + 8 + 1 = 16
        A   = 4'b0111; B   = 4'b1000;cin = 1'b1;  #10;

    end
endmodule
```

## 4X3 MULTIPLIER



```
module MUL_4X3 (A, B, P);
    input  [3:0] A;
    input  [2:0] B;
    output [6:0] P;
    // Partial products
    wire a0b0, a1b0, a2b0, a3b0;
    wire a0b1, a1b1, a2b1, a3b1;
    wire a0b2, a1b2, a2b2, a3b2;

    wire s0, s1, s2, s3;
    wire c0, c1;
    and (a0b0, A[0], B[0]);
    and (a1b0, A[1], B[0]);
    and (a2b0, A[2], B[0]);
    and (a3b0, A[3], B[0]);

    and (a0b1, A[0], B[1]);
    and (a1b1, A[1], B[1]);
   and (a2b1, A[2], B[1]);
   and (a3b1, A[3], B[1]);

   and (a0b2, A[0], B[2]);
   and (a1b2, A[1], B[2]);
   and (a2b2, A[2], B[2]);
   and (a3b2, A[3], B[2]);

    assign P[0] = a0b0;

    // RCA(A, B, Cin, Sum, Cout)
    RCA_4 RCA1 ({1'b0, a3b0, a2b0, a1b0},{a3b1, a2b1, a1b1, a0b1},
               1'b0,{s3, s2, s1, P[1]},c0);
    RCA_4 RCA2 ({c0, s3, s2, s1},{a3b2, a2b2, a1b2, a0b2}, 1'b0,
        {P[5], P[4], P[3], P[2]}, c1);

    assign P[6] = c1;
endmodule
```

```
// 23BEC0137 KAARMUKILAN A G
`timescale 1ns/1ps
module MUL_4X3_tb;

    reg  [3:0] A;
    reg  [2:0] B;
    wire [6:0] P;

    MUL_4X3 dut (A, B, P);

    initial begin
        // 3 × 5 = 15
        A = 4'b0011; B = 3'b101; #10;

        // 7 × 6 = 42
        A = 4'b0111; B = 3'b110; #10;
    end

endmodule
```

```
module MUL_4X3 (A, B, P);
    input  [3:0] A;
    input  [2:0] B;
    output [6:0] P;
    // Partial products
    wire a0b0, a1b0, a2b0, a3b0;
    wire a0b1, a1b1, a2b1, a3b1;
    wire a0b2, a1b2, a2b2, a3b2;

    wire s0, s1, s2, s3;
    wire c0, c1;
    and (a0b0, A[0], B[0]);
    and (a1b0, A[1], B[0]);
    and (a2b0, A[2], B[0]);
    and (a3b0, A[3], B[0]);

    and (a0b1, A[0], B[1]);
    and (a1b1, A[1], B[1]);
   and (a2b1, A[2], B[1]);
   and (a3b1, A[3], B[1]);

   and (a0b2, A[0], B[2]);
   and (a1b2, A[1], B[2]);
   and (a2b2, A[2], B[2]);
   and (a3b2, A[3], B[2]);
     assign P[0] = a0b0;
     // RCA(A, B, Cin, Sum, Cout)
    RCA_4 RCA1 ({1'b0, a3b0, a2b0, a1b0},{a3b1,
a2b1, a1b1, a0b1},
                  1'b0,{s3, s2, s1, P[1]},c0);
    RCA_4 RCA2 ({c0, s3, s2, s1},{a3b2, a2b2,
a1b2, a0b2}, 1'b0,
        {P[5], P[4], P[3], P[2]}, c1);


    assign P[6] = c1;
endmodule

// 23BEC0137 KAARMUKILAN A G
`timescale 1ns/1ps
module MUL_4X3_tb;

    reg  [3:0] A;
    reg  [2:0] B;
    wire [6:0] P;

    MUL_4X3 dut (A, B, P);

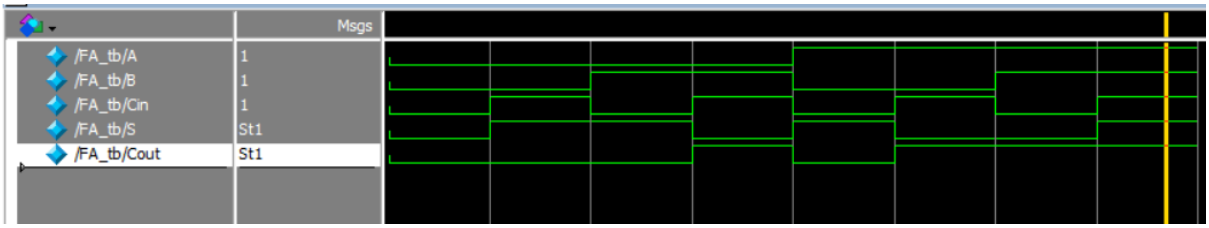
    initial begin
        // 3 × 5 = 15
        A = 4'b0011; B = 3'b101; #10;

        // 7 × 6 = 42
        A = 4'b0111; B = 3'b110; #10;
    end

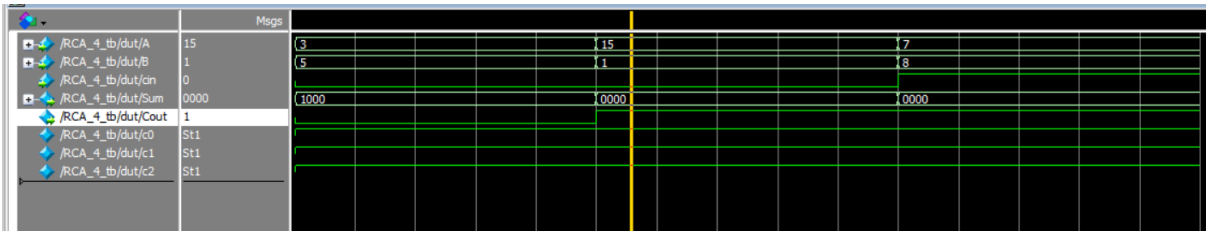
endmodule
```

## FUNCTIONAL SIMULATION:

### FULL ADDER

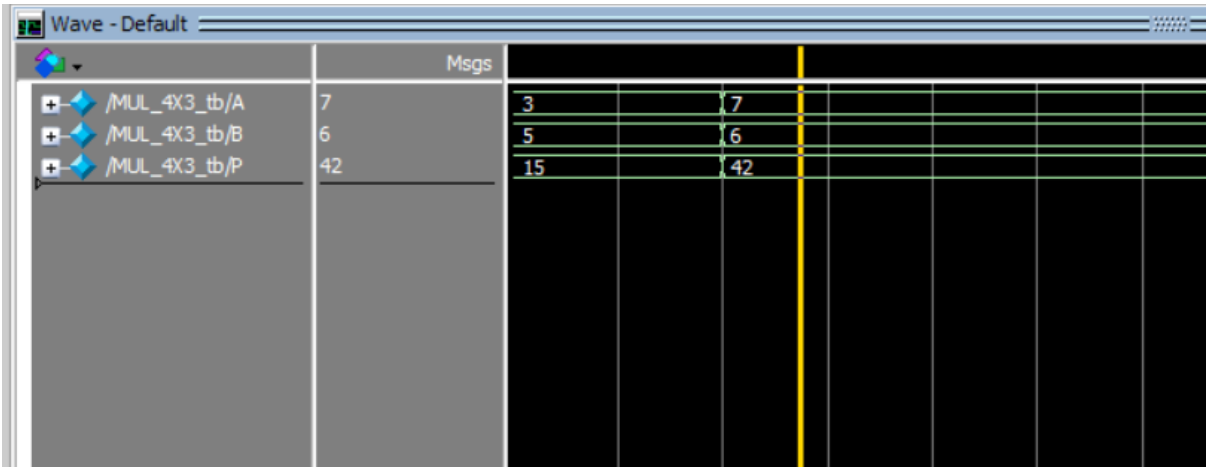



### RCA_4BIT



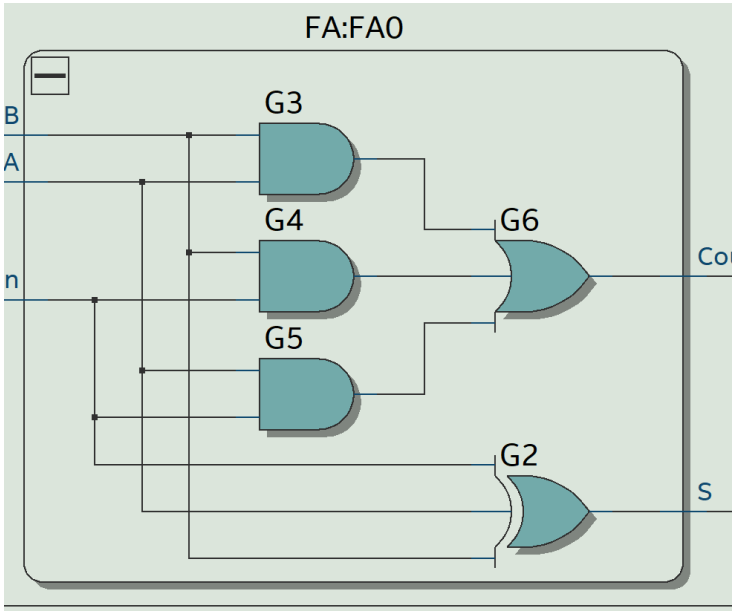

$(15+1 = 7+8+1= (16)_{10} = (1\ 0000)_2)$

### 4X3 MULTIPLIER

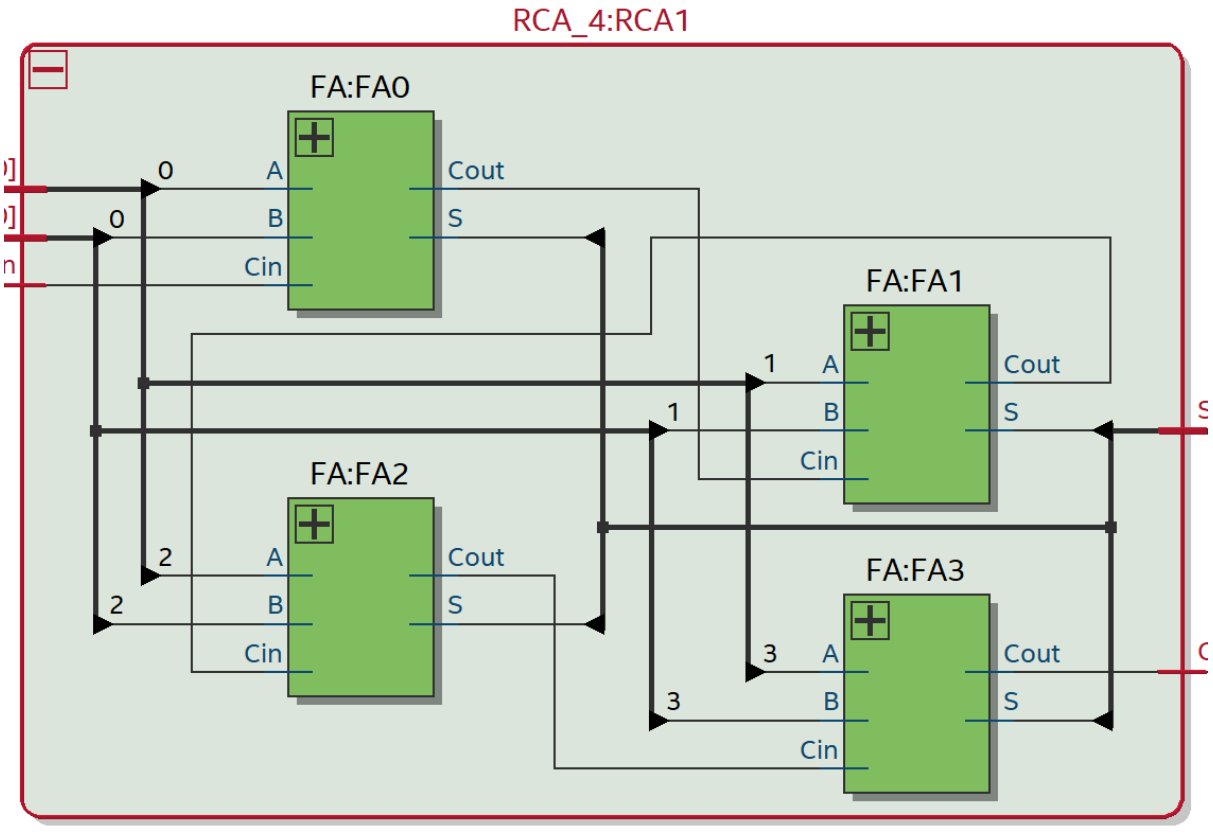


## RTL SCHEMATICS:

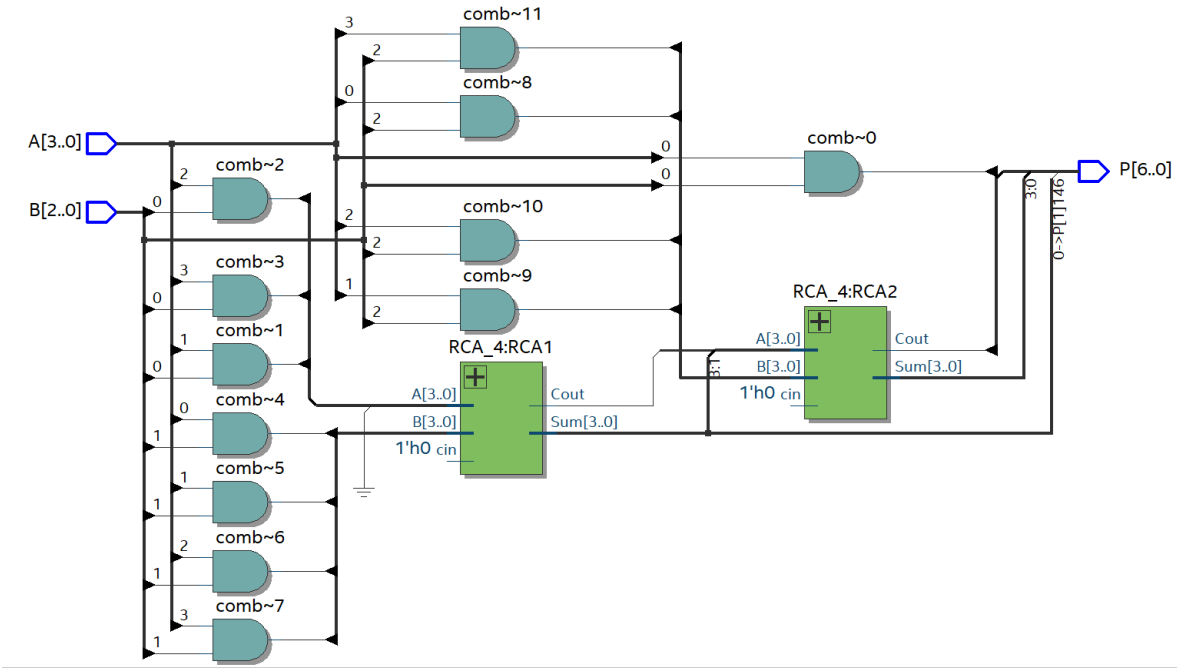
### FULL ADDER



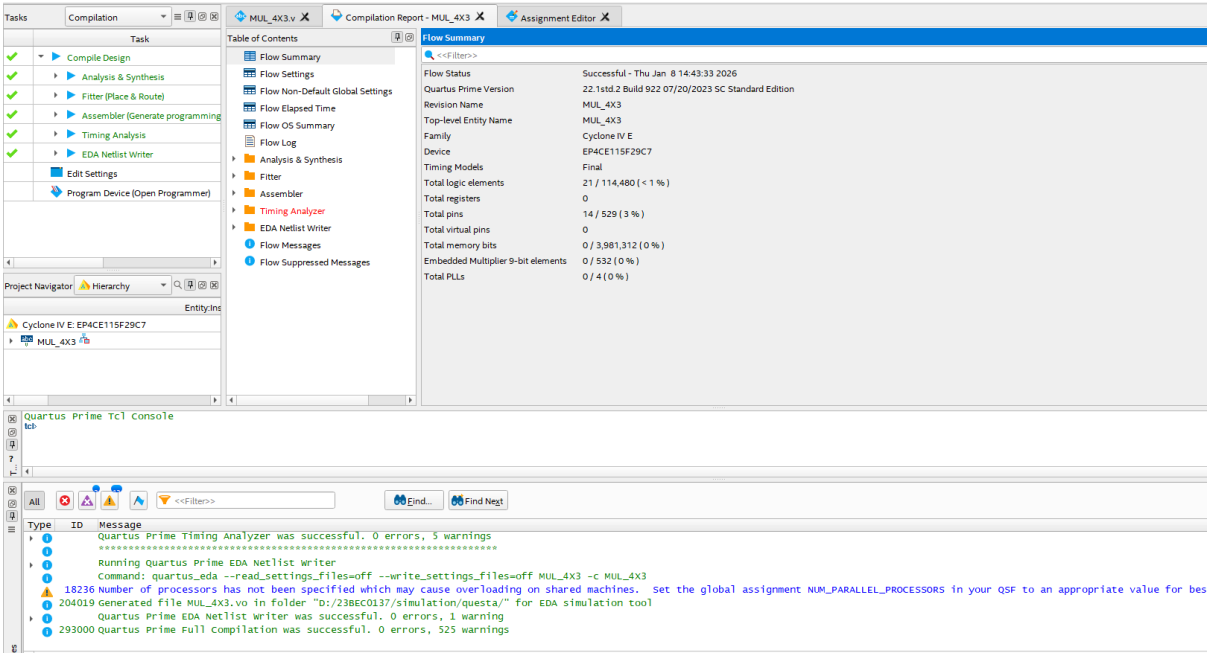

### RCA_4BIT

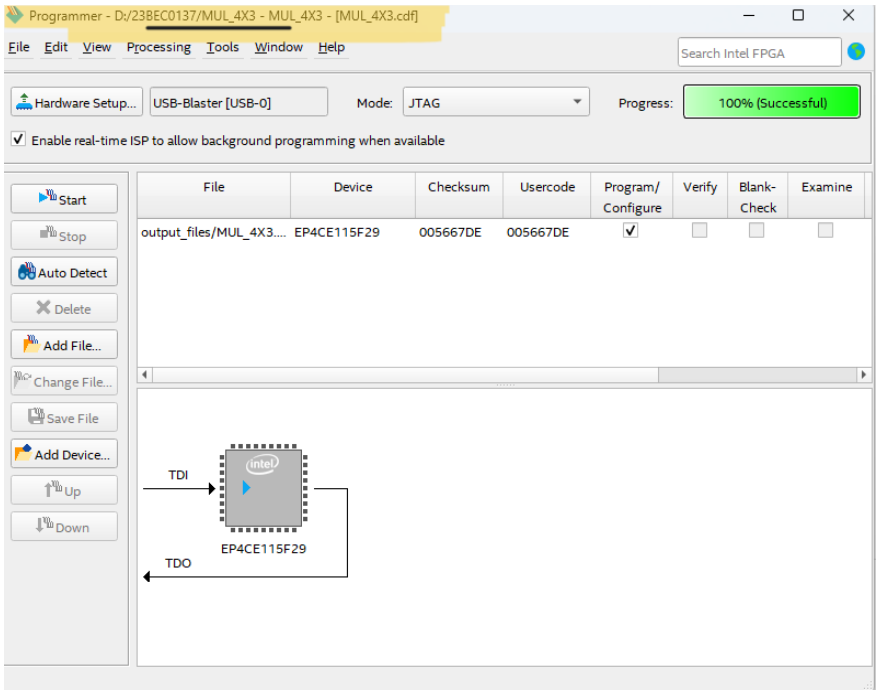


## 4X3 MULTIPLIER

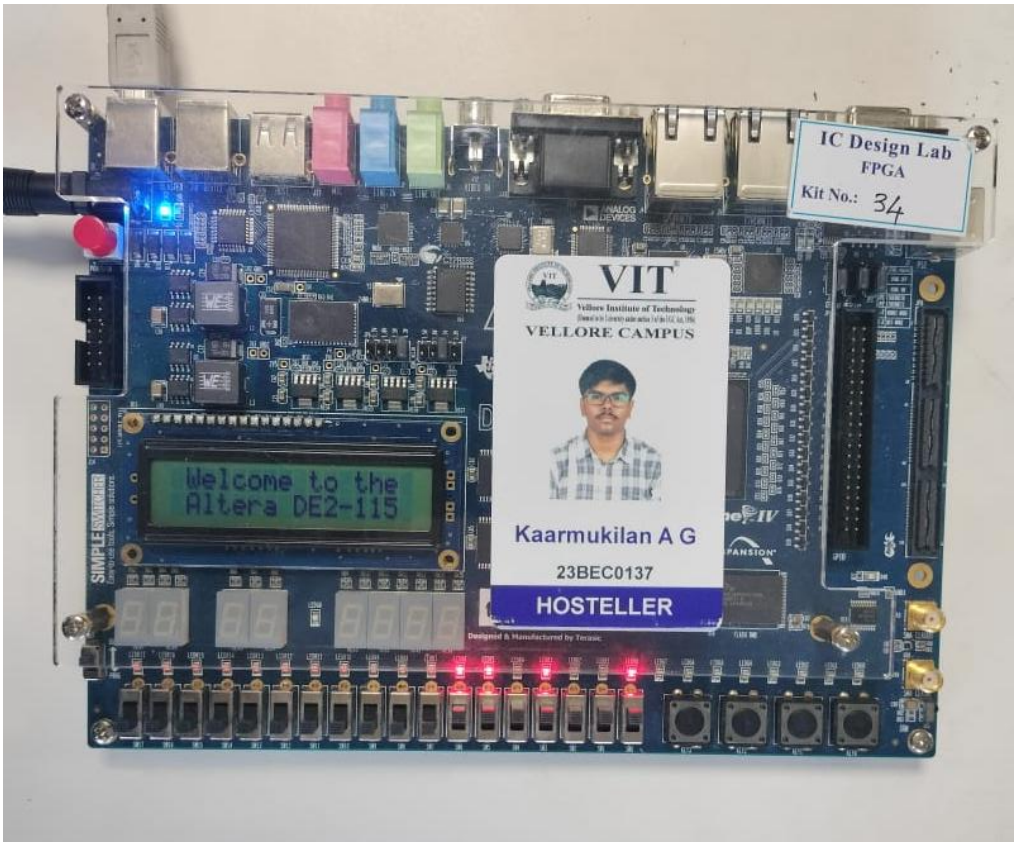



## SYNTHESIS REPORT SUMMARY






## FPGA IMPLEMENTATION:




**7X15=105 (111 X 0011 = 110 1001)**

## INFERENCE:

Hence, 4X3 Multiplier is simulated and verified.

# 4 BIT SEQUENTIAL MULTIPLIER (LAB)

## RTL CODE AND TESTBENCH:

```
module SEQ_MUL (
    input        clk,
    input        rst,
    input  [3:0] A,
    input  [3:0] B,
    output reg [7:0] Z
);
    reg [3:0] M;
    reg [3:0] Acc;
    reg [3:0] Q;
    reg       C;
    reg [3:0] count;
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            Acc   <= 0;
            Q     <= B;
            M     <= A;
            C     <= 0;
            count <= 0;
            Z     <= 0;
        end
        else if (count < 4) begin

            if (Q[0])
                {C, Acc, Q} <= ({1'b0, Acc, Q} + {1'b0, M, 4'b0}) >> 1;
            else
                {C, Acc, Q} <= {1'b0, Acc, Q} >> 1;

            count <= count + 1;
        end
        else begin
            Z <= {Acc, Q};
        end
    end
endmodule

//23bec0137 KAARMUKILAN A G
```

```
//23bec0137 KAARMUKILAN A G
`timescale 1ns/1ps

module tb_SEQ_MUL;
    reg clk;
    reg rst;
    reg [3:0] A;
    reg [3:0] B;
    wire [7:0] Z;
    SEQ_MUL dut (clk,rst, A,B,Z);

    always #5 clk = ~clk;
    initial begin
              clk = 0;
        A   = 0;
        B   = 0;
        #5 rst=1;
        A = 4'd3;
        B = 4'd5;
        #20 rst = 0;
        #100;//4 clk cycle
        rst = 1;
        A = 4'd7;
        B = 4'd9;
        #10 rst = 0;
        #100;

        $stop;
    end

endmodule
```

```
module SEQ_MUL (
    input       clk,
    input       rst,
    input  [3:0] A,
    input  [3:0] B,
    output reg [7:0] Z
);
    reg [3:0] M;
    reg [3:0] Acc;
    reg [3:0] Q;
    reg       C;
    reg [3:0] count;
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            Acc   <= 0;
            Q     <= B;
            M     <= A;
            C     <= 0;
            count <= 0;
            Z     <= 0;
        end
        else if (count < 4) begin

            if (Q[0])
                {C, Acc, Q} <= ({1'b0, Acc, Q} + {1'b0,
M, 4'b0}) >> 1;
            else
                {C, Acc, Q} <= {1'b0, Acc, Q} >> 1;


            count <= count + 1;
        end
        else begin
            Z <= {Acc, Q};
        end
    end
endmodule

//23bec0137 KAARMUKILAN A G
`timescale 1ns/1ps

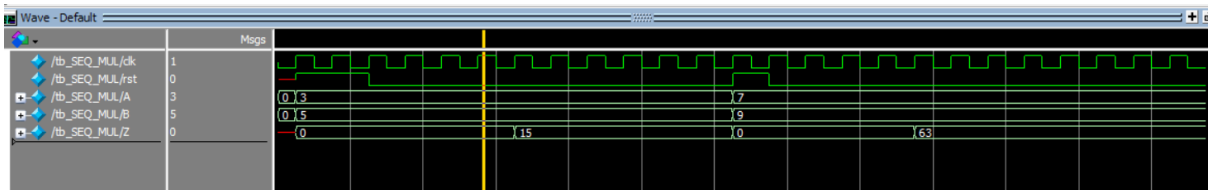
module tb_SEQ_MUL;
    reg clk;
    reg rst;
    reg [3:0] A;
    reg [3:0] B;
    wire [7:0] Z;
    SEQ_MUL dut (clk,rst, A,B,Z);

    always #5 clk = ~clk;
    initial begin
            clk = 0;
        A   = 0;
        B   = 0;
            #5 rst=1;
            A = 4'd3;
        B = 4'd5;
        #20 rst = 0;
        #100;//4 clk cycle
        rst = 1;
        A = 4'd7;
        B = 4'd9;
        #10 rst = 0;
        #100;

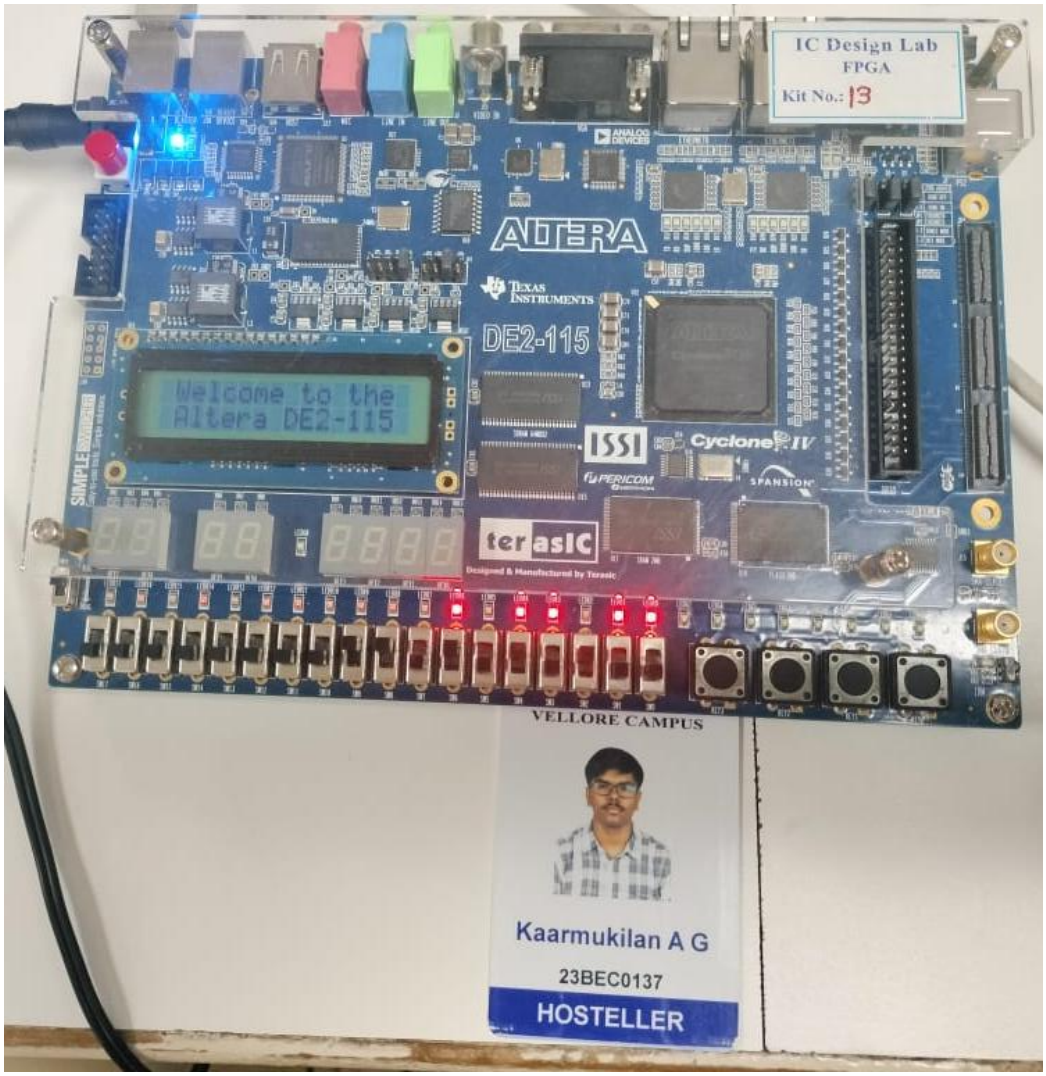

        $stop;
    end

endmodule
```

## FUNCTIONAL SIMULATION:




## FPGA IMPLEMENTATION:




**(13 X 7 =91) (1101 X 0111 =0101 1011)**